OptiKeep System Design

Purpose and document control

OptiKeep system design and requirements specification

Version 1.2 | 26 October 2026 | Prepared for Jason Froebe

Status: implementation baseline proposal. This document defines intended behavior, not a claim of implemented or tested functionality. PostgreSQL and SQLite3 are mandatory supported alternatives. Both provide identical evidence, safety and recovery semantics; backend selection changes database operations and capacity, not detector coverage.

OptiKeep is a Linux and Unraid appliance for distributed photo and video duplicate analysis, quality ranking, explainable keeper selection and reversible quarantine. A single authoritative coordinator owns discovery, persistence, scheduling and all source modifications. Analysis daemons have read-only source mounts and no database credentials.

Scope includes mandatory Czkawka alongside internal detection, FFmpeg and FFprobe, persistent caches, multiple daemons, a web interface, REST API, metrics, Docker packaging and an Unraid template. Windows and macOS qualification are excluded. Testing uses authorized existing local Linux infrastructure with isolated disposable resources and synthetic media. No production libraries are test inputs.

Revision history: 1.0 establishes the greenfield architecture, mandatory dual database support, bounded scanning pipeline and full-analysis safety barrier. No prior implementation is assumed. The work-session requirements remain the detailed feature inventory; this specification resolves architecture and implementation contracts.

Document ownership: repository maintainers own requirements, algorithm versions and release evidence. Keep Markdown, DOCX and PDF versions of this specification and its companion dataflow specification together under docs/design and docs/dataflow. Update revision history and traceability in the same change as architecture, schema, API, algorithms, safety, configuration or concurrency changes.

Release status must distinguish designed, implemented, tested and qualified. Do not call a design acceptance criterion satisfied until a linked test or benchmark demonstrates it.

Architecture and resource admission

D1 Coordinator architecture

Use a Python service with a typed repository layer, versioned schema migrations, an HTTP API and a frontend served by the coordinator. Separate scan orchestration, analysis scheduling, evidence reconciliation, matching, quality ranking and filesystem actions into modules. A local daemon can provide single-container operation without changing the daemon protocol. Persistent worker pools change admitted concurrency rather than being recreated on every scaling decision.

Run one active coordinator in the initial release for either backend. Acquire an exclusive local process lock for SQLite3 and a database advisory lock for PostgreSQL; reject a second coordinator. PostgreSQL availability does not imply supported active-active coordination. Only the coordinator executes source mutations, through one journaled action executor.

D2 Bounded scan pipeline

Stream directory entries with scandir-style traversal. Persist a paginated run manifest and bounded cache lookup batches; never accumulate the entire tree or candidate graph in RAM. Exclude quarantine, cache and application data. Reject source symlinks by default; do not follow links outside allowed roots. Track aliases and hard links without treating them as independent reclaimable storage.

Proposed initial limits, subject to benchmark tuning: 512 manifest rows per commit, 128 file references per task batch, 32 MiB maximum decoded task payload, 256 pending batches, 512 MiB analysis memory per worker, and one large-file reader per configured storage contention domain. Oversized media must use a separately budgeted route or fail visibly; never downsample silently to fit. Apply process and subprocess memory limits in addition to queue bounds.

Keep discovery, hashing, decoding, Czkawka, RPC and database admission budgets separate but charge all readers to shared storage budgets. Account for native decoder threads to avoid nested oversubscription. Unknown Unraid topology uses conservative shared limits; separate mounts or daemons do not establish independent disks. Preserve a canonical share namespace for every mutation.

Scale every 15 seconds with a 60-second cooldown and three stable samples before increasing admission. These are starting defaults, not measured optimal values. Increase only when throughput improves and memory, storage latency and CPU budgets permit; back off on pressure or diminishing returns. Bound preview generation and dashboard queries independently.

Database contract and durable model

D3 Backend selection

Configure database.backend as sqlite3 or postgresql. SQLite3 is the default for a standalone installation; PostgreSQL is selectable at install time or through an offline verified migration. Never silently fall back to an empty backend when connection fails. Daemons always send results to the coordinator, including when many daemons use a SQLite3 coordinator.

SQLite3 uses a database on a local filesystem with reliable locking, WAL mode, foreign_keys enabled on every connection, synchronous FULL, bounded busy_timeout and one coordinator write queue. Use short transactions, bounded retry with jitter and explicit BEGIN IMMEDIATE for claims. Monitor checkpoint progress, WAL size and long readers. Reject network database mounts; keep WAL and SHM beside the database. Prefer a directly mounted local app-data directory for Unraid. SQLite WAL supports concurrent readers but only one writer [S1].

PostgreSQL uses bounded connection pools, short transactions, row locks and backend-specific bulk upserts or COPY staging when measured beneficial. Queue claims may use SKIP LOCKED; never use that mechanism for safety completeness checks. Connection loss pauses scheduling and actions. A database outage must preserve cache files and evidence, not trigger invalidation.

D4 Logical schema

libraries stores allowed roots, storage domains and policy revisions. files stores canonical relative path, library ID and current generation, unique per library/path. file_generations stores device/inode observations, size, nanosecond timestamps and content signature. scan_runs and run_files bind each immutable run to observed generations and inclusion/exclusion outcomes.

analysis_records stores file generation, detector/version, configuration digest, input signature, status and measurements; enforce uniqueness on that cache key. tasks stores requirements, attempt, lease token, expiry and state. evidence_edges stores normalized file pairs, detector provenance, raw distance and verification state. groups, group_members and decisions store run-scoped membership, keeper, policy version and explanations.

plans and plan_actions store manifest revision, evidence digest, source and destination, keeper generation, action intent and recovery state. quarantine_items stores original path, stored path, content digest, retention and restore status. settings_revisions, audit_events, daemon_status and schema_versions complete the durable model. Add foreign keys and indexes on run membership, task state/expiry, cache keys, evidence endpoints and action state.

Use portable UUID text keys and UTC integer microsecond timestamps at the logical boundary; PostgreSQL may use native UUID/TIMESTAMPTZ and JSONB, SQLite3 validated JSON text and integer timestamps. Store search-critical fields as indexed columns. Parameterize SQL and test backend adapters against identical logical fixtures.

Migrations require exclusive maintenance mode and a verified backup. Backend transfer exports versioned logical records, preserves IDs, checks counts, hashes and relationships, then runs read-only validation before enabling actions. No hot backend switching. SQLite backups use its backup API; PostgreSQL uses a consistent database backup. Test restore and pending-action recovery on both.

Analysis cache and duplicate decisions

D5 Cache and identity

A cache key combines file generation, input signature, detector version, algorithm version and relevant configuration digest. Metadata equality is a trust heuristic, not proof of identical bytes. Default conservative mode reuses analysis only under the documented controlled-library assumption with stable identity and change tracking; uncertain identity requires content verification. Strict full-revalidation mode rehashes inputs. Targeted refresh invalidates only dependent records. Preserve valid caches across restarts and daemon outages.

Observe identity before and after reading. Changed, missing or unstable files invalidate the task result and block run completion until reconciled. Filesystem notifications accelerate discovery but never establish scan completeness. Reconcile the final manifest before sealing it; additions or modifications create new analysis work. Continually changing libraries may require a write-free maintenance window or filesystem snapshot. Completion describes a manifest generation, not a globally atomic live filesystem snapshot.

D6 Independent detectors and matching

Use whole-file SHA-256 for exact evidence; size and partial hashes only filter candidates. Compute compatible digests in a streaming read. Decode each image once per compatible task for pHash, dHash, aHash, wHash, metadata and quality measurements. Probe video once and reuse versioned sample frames. Preserve alpha, color profiles and orientation policy in cache signatures. Corrupt or unsupported media has an explicit outcome and cannot silently become successfully analyzed.

Pin and qualify Czkawka, FFmpeg and FFprobe before implementation release. Validate actual CLI capabilities and output fixtures, including similar-video prerequisites, formats, cache behavior and thread controls. Czkawka runs without deletion options and returns independently attributed evidence. Missing required capability degrades readiness and blocks quarantine. No exact version is asserted qualified by this design.

Generate candidates using persisted exact indexes, perceptual BK-trees or validated alternatives and duration/aspect buckets for videos. Partition boundaries must preserve match recall. Oversized buckets spill to persisted work; never truncate silently. Match new files against the full applicable corpus. A connected component alone does not prove every member duplicates its keeper: verify each proposed removal directly against that keeper to avoid similarity-chain errors.

D7 Quality and confidence

Keep versioned raw measurements, normalized components, totals and uncertainty. Image starting weights are sharpness 30, compression 20, retained detail 15, blocking 10, dynamic range 8, color 7, bit depth 5, originality 3 and file-size signal 2 percent, with bounded noise and ringing penalties. Calibrate on synthetic ground truth before enabling automatic decisions. Video ranking considers completeness, effective detail, artifacts, dynamic range, audio and integrity; largest size or resolution alone never wins.

Quality-equivalent copies use oldest trustworthy capture time, then filesystem time with provenance and deterministic path tie-break. Preserve distinctive metadata, subtitles, audio tracks, HDR variants and sidecars unless policy explicitly permits loss. Exact, VERY_HIGH, HIGH, MEDIUM and LOW are rule versions, not probabilities. Correlated perceptual hashes do not count as independent votes. Default quarantine threshold HIGH is ineffective until calibrated; automatic execution remains off initially.

Safety actions and recovery

D8 Global safety barrier

Run states are CREATED, DISCOVERING, ANALYZING, RECONCILING, MATCHING, RANKING, PLANNED, READY, EXECUTING and COMPLETE. FAILED, CANCELLED and INCOMPLETE never authorize execution. Read-only stages can overlap, but READY requires a sealed manifest, no unresolved required tasks/errors, complete independent detector coverage, persisted comparisons, resolved groups, all keeper decisions and a complete durable action plan. Every selected library participates in the barrier.

Snapshot configuration and algorithm versions per run. A policy or input change invalidates the relevant plan. Unsupported files may be explicitly excluded before sealing under a visible policy; failed required analysis cannot be retroactively relabeled excluded just to pass the barrier. Dry-run may create plans and diagnostics but never source actions. Graceful stop during analysis marks the run incomplete and prevents actions.

D9 Filesystem action contract

Default to quarantine; permanent deletion is disabled. Before every action, revalidate both keeper and duplicate generation and content evidence, allowed roots, canonical resolved paths, free space and action intent. Reject symlink substitution, path traversal and destination collisions. Use directory-relative, no-follow operations where available. Because POSIX identity checks do not prevent an unrelated writer from changing an open file, require a quiescent source window or protected immutable snapshot for automatic mutation; otherwise remain in review/dry-run mode.

Commit action intent before filesystem mutation. Prefer atomic no-overwrite rename to quarantine on the same filesystem. For cross-filesystem quarantine, copy to a unique temporary destination, fsync, verify full content, atomically publish, fsync destination directory, and only then unlink the revalidated source and fsync its parent. Never overwrite. Do not mix Unraid user-share and disk paths. Hard-linked paths need explicit policy and accurate reclaimed-byte accounting.

Database and filesystem commits are not atomic together. Record intent, destination verified, source removed and committed states. On restart, reconcile observed source/destination digests with the action journal. Ambiguous recovery stops actions for review; retry must never create another deletion. Database failure during an action halts new actions and leaves recoverable intent.

Restore validates the quarantine item, reserves the original path and refuses collisions; the user can select an alternate allowed destination. Record provenance and verify content. Retention alone never authorizes purge: purge requires explicit enabled policy and verified quarantine ownership. Keep an audit tombstone. Support item/group/all restore, retention extension and scoped purge with confirmation.

On first SIGINT, stop admission, persist checkpoints and finish only a safe action boundary; terminate subprocesses gracefully, flush diagnostics and exit 130. An unfinished action must reconcile before any new action. A second SIGINT may terminate promptly, leaving journal recovery mandatory.

Service contracts and product operations

D10 Daemon API

Use authenticated TLS and scoped per-daemon credentials. Coordinator endpoints POST /api/v1/daemons/register, POST /api/v1/tasks/claim, POST /api/v1/tasks/{id}/heartbeat and POST /api/v1/tasks/{id}/results support daemon pull operation. Registration includes supported algorithms, tool versions and path-map identifiers. Claims return bounded references, observed identities, required measurements, config digest, attempt and opaque lease token; never arbitrary shell commands.

Result schema includes task ID, lease token, attempt, idempotency key, observed pre/post identity, detector versions, measurements, errors and evidence provenance. Accept a result only against the current task requirements and lease; reject stale/conflicting submissions with 409. An identical retry returns the original acknowledgement without duplicate records. Heartbeat every 15 seconds with an initial 90-second lease; expiry causes a new attempt. These are configurable initial limits. Validate every payload and root-relative mapping before scheduling or accepting evidence.

D11 User API and interface

Provide authenticated run creation/status/stop, config validation, daemon health, duplicate browsing, quality explanations, keeper overrides, plan review/execution and quarantine restore/purge. Mutations require role authorization, CSRF protection for browser cookies and idempotency keys. Pagination uses stable cursors; secrets are write-only. Errors include a stable code and request ID without internal credentials. Publish OpenAPI and reject incompatible API versions.

Dashboard shows stage progress, throughput, cache hits, backend health, detector versions, healthy daemons, reclaimable bytes and barrier status. Review includes previews, measurements, provenance and why a keeper wins. All settings appear in searchable Settings with defaults, effective values, source of override, accessible help and restart requirements. Support light, dark and system themes. Precedence is CLI, environment, persisted settings, defaults; container-owned values are visible read-only. Settings changes are audited with secrets redacted.

D12 Packaging and observability

Ship coordinator/local-daemon and remote-daemon Docker modes, pinned dependency manifests, health checks and non-root application processes. Validate PUID, PGID, UMASK and writable /config, /cache, /logs and quarantine mounts at startup. Remote media mounts are read-only. PostgreSQL credentials use environment secrets or protected configuration; SQLite3 requires no separate database container. Unraid template exposes backend choice, SQLite path or PostgreSQL connection fields and approved icon. Preserve both approved branding masters without regeneration.

Expose /health liveness, /ready capability/readiness and authenticated or separately protected /metrics. Record discovery, decode, hash, Czkawka, RPC, database, candidate and action latency plus queue peaks, cache hits, CPU and RSS. Avoid path labels and high-cardinality identifiers. Structured JSONL includes run/task IDs; redact tokens and sensitive paths in exported reports. Verify third-party licenses and retain the repository license; use MIT only if no prior license determines otherwise.

Acceptance traceability and implementation sequence

D13 Qualification and traceability

R01 dual backends maps to D3-D4: identical repository, migration, backup/restore and crash-recovery tests on SQLite3 and PostgreSQL. Neither backend may be a skipped optional matrix entry.

R02 bounded scans maps to D1-D2: increasing corpus sizes must keep queues and RSS inside configured budgets, including native allocations. Worker sweeps must show saturation and safe backoff.

R03 cache correctness maps to D5: warm trusted unchanged rescans perform zero avoidable internal hashes/decodes; strict-mode verification reads are reported separately. Additions, removals, replacements, coarse timestamps and algorithm changes must invalidate only required dependencies.

R04 independent evidence maps to D6: pinned Czkawka parser fixtures and live capability checks for exact/similar image/video detection; failures must leave readiness degraded and barrier closed.

R05 decisions maps to D6-D7: annotated transforms and unrelated lookalikes test recall, false positives, chain grouping, metadata preservation and quality equivalence. Baseline and optimized paths must produce equivalent safety outcomes.

R06 full barrier maps to D8: failed discovery, unreadable media, expired leases, missing keepers and incomplete groups produce no source mutation. Concurrent changed inputs invalidate plans.

R07 recovery maps to D9: inject interruption before/after each journal, copy, rename, fsync and unlink boundary; restore and retry never overwrite or lose the retained copy.

R08 API and UX maps to D10-D11: authentication, traversal, idempotency, stale attempts, complete settings coverage, keyboard help, themes and responsive review pass automated checks.

R09 deployment maps to D12: fresh mounted directories, UID/GID, permissions failures, both database configurations, daemon read-only mounts, approved template icon and health checks pass Linux container smoke tests.

R10 performance maps to D2 and D5-D6: benchmark cold application cache, warm rescan, incremental changes, restart, high duplication, pathological buckets, multiple daemons and concurrent UI traffic. Record seeded corpus, media mix, bytes, storage type, versions, worker limits and application versus OS cache state. Measure stage/end-to-end duration, bytes read, decode counts, query/RPC counts, candidates, peak memory and p50/p95 latency. Establish numeric regression gates from repeated baselines before optimizing; no universal throughput claim is made.

Implement in order: portable persistence and migrations; read-only bounded manifest/tasks; internal and Czkawka analysis; matching and quality calibration; barrier and journaled actions; UI/API/settings; Docker and Unraid packaging; performance tuning and full qualification. Each milestone must include meaningful tests and aligned design/dataflow revisions. Enforce PEP 8, PEP 257, formatting, static analysis and typed contracts. Use only isolated authorized local Linux testing; preserve unrelated persistent services.

Sources and unresolved qualification

[S1] SQLite Write Ahead Logging: https://www.sqlite.org/wal.html (consulted 25 October 2026). PostgreSQL locking semantics: https://www.postgresql.org/docs/current/sql-select.html. These describe backend mechanisms, not measured OptiKeep performance. Exact dependency versions, SQLite local filesystem validation, perceptual thresholds, video-quality weights and numeric benchmark gates remain implementation qualification items. They must be resolved before enabling automatic quarantine.

Implementation status and qualification evidence

Revision 1.1 | 25 October 2026 | Development implementation 0.1.0

This revision records the first executable read-only implementation. The full architecture in D1-D13 remains the target; its presence in this specification does not mean every component has been delivered.

Implemented modules: optikeep/db.py defines the portable schema and transaction/lease adapter; coordinator.py owns batched discovery, cache admission and result validation; media.py performs internal whole-file hashing, image hashes and initial measurements, and FFprobe metadata extraction; api.py exposes authenticated management and daemon routes; cli.py runs the coordinator or sequential daemon. process.py bounds subprocess output and runtime. ownership.py prevents simultaneous coordinator ownership; storage.py rejects known network filesystem types for SQLite.

SQLite uses WAL, foreign keys, FULL synchronization and immediate write transactions. PostgreSQL uses row-locked task claims and a session advisory ownership lock. Cache reuse is explicitly opt-in and metadata-trusted; strict scans are the default. Exact candidates use an indexed SHA-256 column and bounded review output with a truncation flag. Candidate truncation does not establish completeness and is never actionable.

The current UI provides scan submission, progress, exact-candidate review and redacted read-only bootstrap settings. It is an initial interface, not the complete Settings or review experience in the target specification. Configuration precedence currently covers environment over defaults; persisted editable Settings and broad CLI overrides remain pending.

Safety state: every run reports barrier_open false and mutation_enabled false. No source-action route or executor exists. The application cannot reach READY. Czkawka 12.0.2 is checksum-pinned in the image and its version command has run on a Linux build worker; detector invocation/parsing and independent evidence coverage are pending. Initial image measurements are explicitly not qualified keeper rankings.

Initial isolated qualification: the first functional suite passed 12 tests on SQLite3 and the same 12 on PostgreSQL. The first CI run failed its subsequent static-check step because Ruff attempted to write in the non-root image directory; functional success is not a full CI pass. The corrected final suite adds ownership, heartbeat expiry, subprocess budgets and exact-candidate tests. Final results must be read from the accompanying validation report.

Requirements implementation matrix

Requirement	Implementation	Qualification
R01 dual backends	Schema, leases and results implemented	Foundational tests on both; backup/transfer pending
R02 bounded scans	Batched manifest and bounded subprocess output	Large-corpus memory benchmarks pending
R03 cache correctness	Strict default and opt-in metadata reuse	Change and cache fixtures present; broader trust tests pending
R04 independent evidence	Czkawka binary pinned and runnable	Evidence integration pending
R05 decisions	Internal measurements and exact review	Perceptual grouping, calibration and keepers pending
R06 full barrier	Source actions unavailable; barrier closed	Incomplete-run fixtures present
R07 recovery	No filesystem mutation executor	Quarantine, restore and purge pending
R08 API and UX	Token-scoped initial API and UI	Initial auth fixtures; complete UX qualification pending
R09 deployment	Non-root development Docker and Compose	Linux image built; full Unraid packaging pending
R10 performance	Batched discovery and indexed exact candidates	Baselines and regression budgets pending

No release or production readiness is claimed. Remaining work includes independent Czkawka image/video detectors, internal video fingerprints, indexed perceptual matching, complete quality scoring, group decisions, journaled actions, audit/configuration history, complete settings/UI, autoscaling, metrics, approved branding integration, Unraid packaging, migrations beyond initial schema, backup/restore/backend transfer, distributed qualification and reproducible benchmarks.

Revision 1.2 PostgreSQL template configuration

Version control

Version control entry 26 October 2026. This revision changes the Unraid configuration contract without changing the database schema, daemon API, source media access or read-only safety barrier.

D12 Database connection fields

The coordinator template selects sqlite3 or postgresql through OPTIKEEP_DATABASE_BACKEND. SQLite uses OPTIKEEP_SQLITE_PATH, an absolute local file path defaulting to /config/optikeep.sqlite3. PostgreSQL uses OPTIKEEP_POSTGRES_HOST, OPTIKEEP_POSTGRES_PORT, OPTIKEEP_POSTGRES_DATABASE, OPTIKEEP_POSTGRES_USERNAME and OPTIKEEP_POSTGRES_PASSWORD. The default port is 5432; accepted ports are 1 through 65535. All remaining PostgreSQL fields are required when that backend is selected.

Credential handling and precedence

Users enter PostgreSQL passwords literally. SQLAlchemy constructs and escapes the connection URL, including reserved characters. The Unraid password field is intentionally unmasked at the owner request; API settings responses remain redacted. Workers receive no database credentials. Explicit backend selection takes precedence over OPTIKEEP_DATABASE_URL. The legacy URL is retained only when no backend is explicitly selected. Invalid backends, incomplete PostgreSQL fields and relative SQLite paths fail instead of selecting another backend.

Token guidance

Coordinator administrator and daemon token descriptions include openssl rand -hex 32. Generate distinct administrator and daemon tokens; use the same daemon token on the coordinator and its workers.

Requirements traceability

R01 and R09 map to optikeep/config.py, the coordinator and worker XML templates, and tests/test_core.py. Verification covers literal password and username round trips, port bounds, missing fields, redaction, backend precedence, absolute SQLite paths and unsupported backend rejection. Both backends must pass on the existing Meta Linux environment before the updated image bundle is distributed.